



Week 1 Solution

Question 1

Question

For a 32-bit address bus, which of the following hexadecimal numbers is **not a valid address**?

1. A12345BF
2. 0A12345B
3. A12345B
4. A12345BFF

Answer

A12345BFF

Solution

A 32-bit address contains exactly **32 bits**.

One hexadecimal digit represents **4 bits**.

Therefore:

```
32 bits / 4 bits
= 8 hexadecimal digits
```

So, a valid 32-bit hexadecimal address must contain **at most 8 hexadecimal digits**.

Let's check the options:

A12345BF

Contains 8 hexadecimal digits → valid.

0A12345B

Contains 8 hexadecimal digits → valid.

A12345B

Contains 7 hexadecimal digits → also valid because the leading zero can be omitted.

A12345BFF

Contains **9 hexadecimal digits**:

A 1 2 3 4 5 B F F

Therefore:

$$9 \times 4 = 36 \text{ bits}$$

It cannot be represented using a 32-bit address bus.

Hence:

Correct Answer: A12345BFF

Question 2

Question

Which of the following statements is True?

- A. Instruction Pointer points to the next instruction to be executed.
- B. Stack Pointer points to the base of the stack.

Options:

1. a- true, b- true
2. a- false, b- true
3. a- true, b- false
4. a- false, b- false

Answer

a- true, b- false

Solution

Statement A: Instruction Pointer points to the next instruction

True.

The **Instruction Pointer (IP)** keeps track of where the processor should fetch the next instruction for execution.

Conceptually:

```
Instruction 1
Instruction 2 ← Instruction Pointer
Instruction 3
Instruction 4
```

The processor uses the instruction pointer to determine the next instruction to execute.

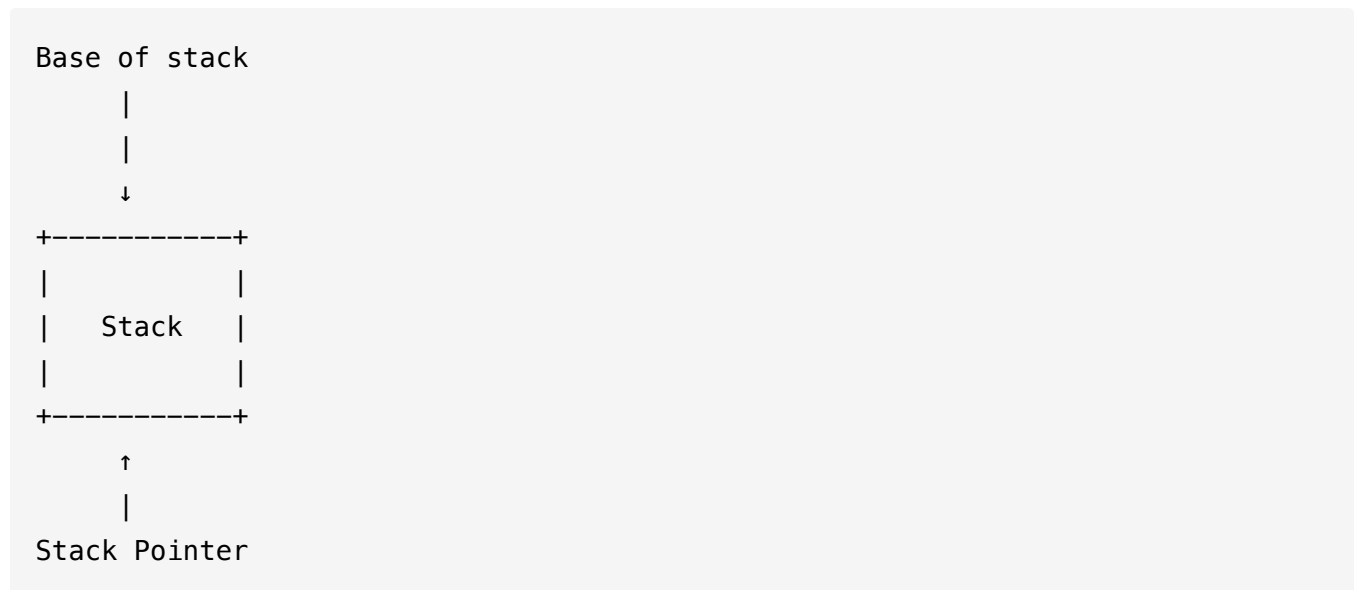
Therefore, statement A is **True**.

Statement B: Stack Pointer points to the base of the stack

False.

The **Stack Pointer (SP)** generally points to the **top/current position of the stack**, not the base.

The stack can be visualized as:



The exact direction in which the stack grows depends on the architecture, but the key point is that the stack pointer tracks the current top of the stack.

Therefore:

A → True
B → False

Correct Answer: a- true, b- false.

Question 3

Question

A backward-compatible Operating System requires all applications to be redesigned.

State whether this is True or False.

Answer

False

Solution

Backward compatibility means that a newer version of an operating system continues to support applications designed for an older version.

The purpose of backward compatibility is precisely to avoid requiring every existing application to be redesigned.

For example:

```
Old Application
    ↓
New Operating System
    ↓
Application continues to work
```

If every application had to be redesigned whenever the OS was updated, the system would not provide effective backward compatibility.

Therefore, the statement is:

False.

Question 4

Question

State True or False.

- A. Kernel can access user address space.
- B. User cannot access Kernel address space.

Options:

1. A - True, B - True
2. A - False, B - False
3. A - True, B - False
4. A - False, B - True

Answer

A - True, B - True

Solution

Statement A: Kernel can access user address space

True.

The kernel needs to interact with user-space memory for various operations.

For example, when a user program passes data to the kernel through a system call, the kernel may need to access that user-space data.

Therefore:

Kernel → User Address Space

is possible, subject to appropriate protection and validation.

Statement B: User cannot access Kernel address space

True.

The kernel operates with higher privileges than normal user programs.

User programs should not be able to freely access kernel memory because that would allow them to:

- modify kernel data,
- execute privileged code,
- compromise the operating system.

Therefore, memory protection mechanisms prevent ordinary user-mode code from directly accessing protected kernel memory.

Conceptually:

```
User Mode
  |
  | x Direct unrestricted access
  ↓
Kernel Memory
```

Thus:

```
A → True
B → True
```

Correct Answer: A - True, B - True.

Question 5

Question

TRAP always jumps to a fixed address.

State True or False.

Answer

True

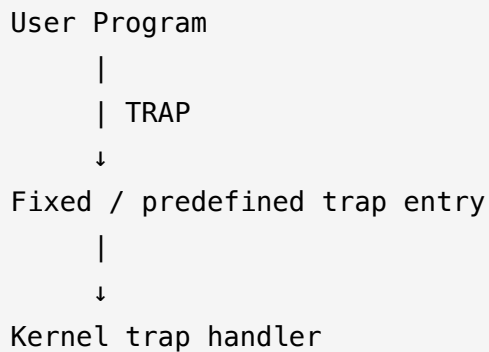
Solution

A **trap** transfers execution from the current program to a predefined kernel handler.

The processor needs to know where the appropriate trap-handling code is located.

Therefore, the trap mechanism uses a predetermined entry point/address associated with the trap handling mechanism.

Conceptually:



This allows the operating system to regain control in a controlled manner whenever a trap occurs.

Therefore, according to the assignment:

True.

Question 6

Question

MS-DOS is a non-monolithic OS structure.

State True or False.

Answer

False

Solution

A **monolithic operating system** has most major OS services operating within a single large kernel.

MS-DOS is generally considered to have a relatively simple/monolithic structure rather than being a microkernel or non-monolithic architecture.

Therefore, the statement:

MS-DOS is a non-monolithic OS structure

is incorrect.

Hence:

False.

Question 7

Question

When two applications want to access the same resource, it leads to a _____.

1. Race Condition
2. Deadlock
3. Mutual Exclusion
4. None of the above

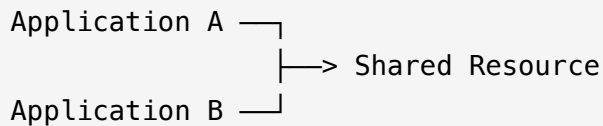
Answer

Race Condition

Solution

A **race condition** occurs when multiple processes or threads access shared data/resource concurrently and the final result depends on the order in which their operations occur.

For example:



Suppose both applications try to modify the same variable:

Initial value = 10

If both read the value before either writes the updated value, the final result may depend on which operation happens first.

This unpredictable dependency on execution order is called a **race condition**.

Mutual exclusion

Mutual exclusion is actually a **solution/control mechanism** used to ensure that only one process accesses a critical section at a time.

Therefore, the condition described in the question is:

Race Condition.

Question 8

Question

A System Call changes the execution from User Mode to Kernel Mode.

State True or False.

Answer

True

Solution

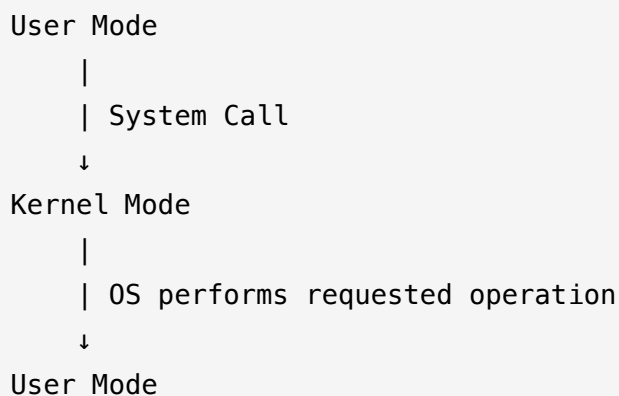
A user program normally executes in **User Mode**, where it has restricted privileges.

When it needs an operating-system service, such as:

- opening a file,
- reading data,
- creating a process,
- allocating certain resources,

it makes a **system call**.

The system call causes a controlled transition:



The transition to kernel mode is necessary because the operating system needs privileged access to hardware and protected resources.

Therefore:

True.

Question 9

Question

Microkernel is likely to have more vulnerabilities than Monolithic Kernel.

State True or False.

Answer

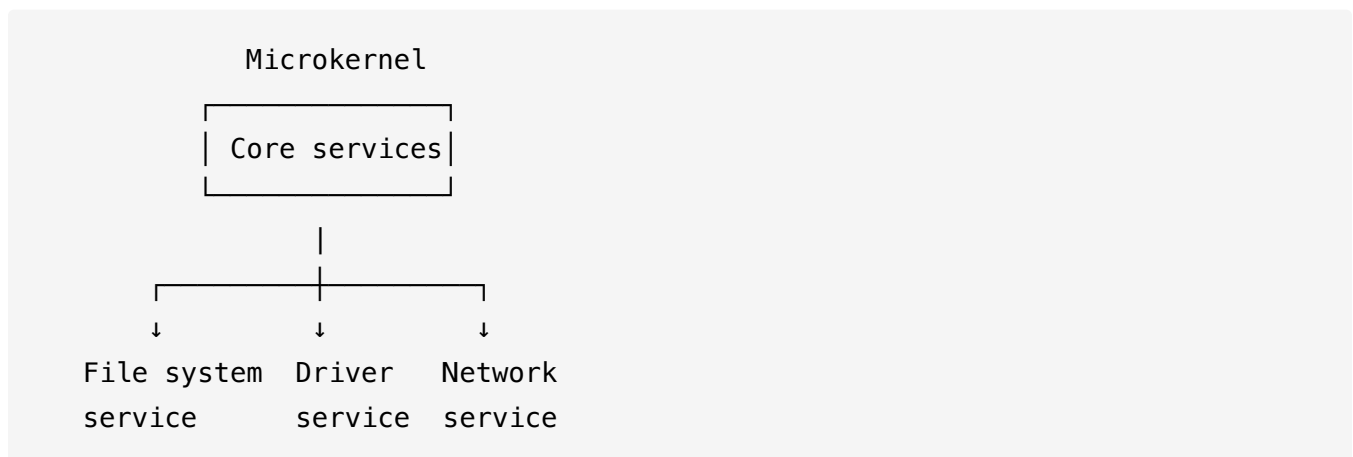
False

Solution

A **microkernel** keeps only essential functionality inside the kernel.

Other services are moved outside the kernel, often into user-space processes.

Conceptually:



A monolithic kernel, on the other hand, generally contains many OS services inside the kernel itself.

Because a microkernel has a **smaller trusted kernel**, the amount of privileged kernel code is reduced. This can reduce the potential attack surface within the kernel.

Therefore, the statement:

Microkernel is likely to have more vulnerabilities than Monolithic Kernel.

is **False**.

Correct Answer: False.

Question 10

Question

Heap section of memory is mainly used for dynamic memory allocation.

State True or False.

Answer

True

Solution

The **heap** is the part of a process's memory used for dynamically allocated memory.

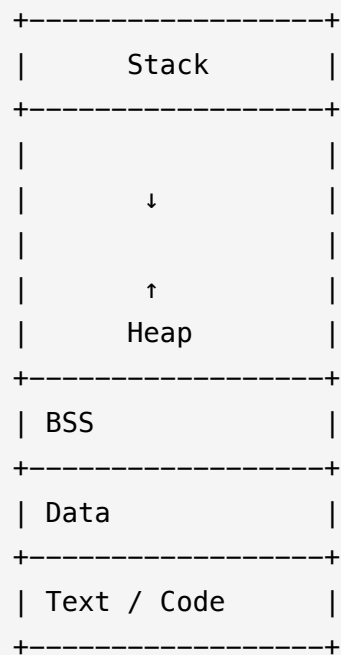
For example, in C:

```
int *p = malloc(100 * sizeof(int));
```

The memory requested by `malloc()` is allocated dynamically from the heap.

A simplified process memory layout is:

High Address



Low Address

The heap can grow as the program requests more dynamically allocated memory.

Therefore:

True.